

实验二：在线评测系统（OJ）实验报告

程序设计训练（Python） · 2026 夏季学期

一、系统功能与设计

1.1 系统架构

系统分三层：

- 前端：Streamlit
单页应用，负责展示题目、提交代码、查看评测结果。前端只做交互和展示，不参与权限判断。
- 后端：FastAPI，全部接口用 `async def` 实现，负责鉴权、评测调度、数据读写。
- 存储：题目用 JSON（一题一文件），用户、提交、日志、AI 任务用 SQLite（同一个 `data/oj.db`）。

前后端通过 HTTP 通信，登录态用 session cookie 维护。后端接口统一返回 `{code, msg, data}`，HTTP 状态码和 code 一致。

1.2 模块划分

后端 `app/` 下分两块：

- `app/core/`：存储层和核心逻辑。`storage.py` 管题目 JSON，`user_store.py` / `submission_store.py` / `access_log_store.py` / `ai_store.py` 管 SQLite，`judge.py` 是评测沙箱，`llm.py` 调大模型，`auth.py` 是鉴权依赖。
- `app/routers/`：路由层。`problems.py`（题目）、`languages.py`（语言）、`submissions.py`（提交）、`users.py`（用户）、`logs.py`（日志）、`ai.py`（AI 命题）、`system.py`（重置）。

前端 `frontend/` 下五个文件：`app.py`（入口和导航）、`views.py`（各页面渲染）、`theme.py`（设计系统）、`i18n.py`（中英文案）、`api_client.py`（HTTP 封装）。

1.3 主要功能

- 题目管理：增删改查、字段校验、删除题目时级联清理提交和日志。
- 评测控制：Python / C++ 两种语言，支持动态注册新语言，带时间/内存限制和输出比对，判定 AC/WA/RE/TLE/CE/MLE。
- 评测管理：提交记录查询、状态查询、重新评测。
- 用户管理：注册、登录、角色管理（admin/user/banned）、用户列表、删除用户。
- 评测日志：测试点明细（AC/WA/耗时/内存）、日志公开与否、访问审计。
- 前端交互：登录页、题目页、做题页、我的页，亮暗双主题和中英文切换。
- AI 命题：配置模型、提交需求、后台异步生成完整题目并入库，实时进度、中断、Token 用量和费用统计。

1.4 技术选型

部分	选型	理由
后端	FastAPI	原生支持 <code>async def</code>

前端	Streamlit	快速搭建交互界面
题目存储	JSON 一题一文件	文档型数据，便于查看和编辑
关系数据	SQLite	零配置，事务简单
评测沙箱	subprocess + psutil	Linux 下限制时间/内存
AI 命题	DeepSeek V4 Flash	OpenAI 兼容接口，便宜够快

二、关键实现与难点

2.1 评测沙箱与资源限制

评测的核心是安全、可控地跑用户代码。用 `subprocess.Popen` 起子进程，另开一个监控线程用 `psutil` 每 20ms 读一次进程内存，超内存就 `kill`；主线程 `communicate(timeout=time_limit)` 超时就判 TLE。

这里有两个细节：内存要取"峰值"，所以监控线程里持续比较并记录最大值；不同语言的运行命令不同，用 `run_cmd` 里的 `{src} / {exe}` 占位符统一抽象，编译型语言先编译再运行。

时间/内存限制的优先级也踩过坑：题目没配限制时应该回退到语言配置，再回退到系统默认（3s / 128MB）。一开始 `Problem` 模型给 `time_limit` 设了 `Pydantic` 默认值 3.0，导致"没配"和"配了 3.0"分不清，语言配置永远不生效。后来改成 `Optional`（None 表示未配置），评测时按"题目 → 语言 → 系统默认"三级回退才正确。

2.2 异步评测

评测是阻塞的（等子进程跑完），如果直接在接口里同步等待会卡住整个事件循环。做法是接口里立刻创建 `pending` 记录并返回，真正的评测用 `asyncio.create_task` 扔到后台，`asyncio.to_thread` 包住同步的 `judge` 函数，跑完再回写 `SQLite`。前端轮询提交状态，看到 `pending` → `success`。

2.3 异常与权限

异常按 401 > 403 > 400 > 429 > 409 > 404 > 500 的顺序判断。用一个 `AppError(code, msg)` 异常类 + 全局异常处理器统一转换成 `JSONResponse`，保证状态码和响应体里的 `code` 一致。校验类错误（FastAPI 默认 422）也统一转成 400。判断顺序体现在各接口里：先鉴权（401/403），再校验参数（400），再频率限制（429），再冲突（409），最后查不存在（404）。

权限上做了一些防护：管理员不能修改或删除其他管理员，也不能删除自己；初始管理员账号不可删；日志在未公开时，提交者本人也只能看到总分，看不到测试点明细。

2.4 Streamlit 主题定制

这是花时间最多的地方。`Streamlit 1.63` 的 `widget` 内部结构跟文档写的不完全一样：`st.pills` 和 `st.segmented_control` 前端其实是同一个 `ButtonGroup` 组件（`testid` 是 `stButtonGroup` 而不是 `stPills`）；输入框的背景色在内层 `stTextInputRootElement` / `stNumberInputContainer` 上，而不是外层 `testid` 元素；图标是 `Material Symbols Rounded` 字体 `ligature`，字体没加载就 fallback 成乱码文字。

最后是直接翻 Streamlit 编译后的 JS 源码，找到真实的 testid 和 DOM 嵌套，再写 CSS 覆盖。中间反复出现"改了没生效"，根因基本都是选择器没对上真实的 testid。

2.5 AI 命题

AI 命题走 OpenAI 兼容接口，把需求喂给 DeepSeek，让它只输出一个 JSON（题目字段 + 测试点）。关键是让模型输出稳定、可直接评测：系统提示里明确要求"≥3 个测试点、覆盖边界、数据规模能区分不同复杂度、不依赖第三方库"，输出后剥掉代码块围栏和前后废话，再做 JSON 解析。任务用后台异步跑，多阶段进度，中断是软中断（标记 cancelled，LLM 返回后不写库）。Token 用量从响应里的 usage 解析，费用按配置的单价算。

三、成果展示

边界测试结果（均实测通过）：

提交	预期	结果
a,b=map(int,input().split());print(a+b)	AC	10/10
print(0)	WA	0/10
1/0	RE	0/10
while True: pass	TLE	0/10（超时）
C++ 语法错误	CE	编译错误信息
同题 1 分钟提交超 3 次	429	频率超限

四、AI 使用说明

本项目采用 Vibe Coding 的方式开发，核心是人和 AI 的一轮轮交互：

- 人提需求：描述要什么功能，比如"做题页要能上传代码文件""提交后清空代码区"。
- AI 实现：AI 写出对应代码，有时会给出多个方案让选择（比如数据存储方式、鉴权方案）。
- 人验收：在浏览器里实际操作，发现问题就截图反馈。
- AI 定位修复：AI 根据截图和描述定位根因，改完再交给验收，循环到符合预期。

交互中最典型的是 Streamlit 主题定制那一段：改了很多轮 CSS 都没生效，AI 最后翻 Streamlit 编译后的 JS 源码，才发现选择器用的是不存在的 testid。还有题目必填字段、日志可见性这类设计决策，是人看了 FAQ 答疑后拍板的，AI 照着实现。

工具链是 WorkBuddy（对话式编程助手）负责写代码、排查、验证；DeepSeek V4 Flash 负责 AI 命题模块的题目生成。代码绝大部分由 AI 生成，人在其中负责需求、设计决策和验收。

五、总结与建议

这个项目把 FastAPI、异步、数据库、前后端串到了一起，完整走了一遍"设计 → 实现 → 反复验收 → 修 bug"的流程。评测沙箱和权限体系的设计能让人理解 OJ 系统背后的复杂度，Streamlit 主题定制则体现了"文档和实际实现有差距"时得去翻源码找真相。

可以改进的地方：评测沙箱目前只限制时间和内存，没有限制输出大小和系统调用，安全性有限；AI 命题的中断还是软中断，没法真正打断正在进行的 LLM 请求；做题页的代码编辑器在切换主题时，编辑器主题不会自动跟随。

时间投入大约四天，其中前端主题定制和 AI 命题的打磨占了一半以上时间。